

A guide to estimate consumer reliance on basal food sources using bulk stable isotopes

An example with a simulated food web

Mattia Ghilardi

3 July 2026

Table of contents

1	Preamble	1
2	Initial setup	1
3	Data simulation	2
4	Estimating consumer trophic position	7
5	Isotopic rescaling of consumers	13
6	Mixing model validation	14
7	Estimating basal source proportions	16
8	References	25
9	Computational environment	25

1 Preamble

This vignette illustrates step-by-step how to estimate the contribution of basal sources to the diet of consumers at multiple trophic positions using bulk stable isotope data in R. To do this, we simulate carbon and nitrogen stable isotope data for a food web with three sources, two isotopic baselines (i.e. primary consumers), and 20 consumer species from five different families.

2 Initial setup

First, to make the simulated data and results reproducible, we need to select a seed.

```
my_seed <- 9
```

Then, we load some required R packages and custom functions.

```
# Load packages
library(readr) # for reading and writing files
library(tibble) # for using tibbles
library(dplyr) # for data manipulation
library(tidyr) # for tidying data
```

```
library(forcats) # for reordering factors
library(ggplot2) # for plotting data
library(tRophicPosition) # for estimating trophic position
library(MixSIAR) # for fitting Bayesian stable isotope mixing models
library(patchwork) # for combining ggplots
library(ggtext) # for Markdown and HTML rendering for ggplot2
```

i Note

We will be using a few MixSIAR functions that are available in the development version, but not in the latest version published on CRAN (3.1.12), which has not been updated since 2020. To use these functions, install the development version of MixSIAR from [GitHub](#). For all other packages, see Section 9 for the version used.

```
# Load custom functions available as appendices to the manuscript,
# placed here in a subfolder called "R"
source("R/simulate_food_web.R") # Appendix S3
source("R/estimate_tp_error.R") # Appendix S8
```

i Note

To use the custom function `simulate_food_web()`, some additional packages must be installed, but they don't need to be loaded. These include: “rlang”, “purrr”, “compositions”, and “EnvStats”.

3 Data simulation

We simulate the data using the custom R function `simulate_food_web()`, which allows us to simulate isotopic signatures for a food web, including basal sources (i.e. primary producers), up to two isotopic baselines (i.e. primary consumers), and higher-level consumers, organised into species and families. The function has several arguments that give the user flexibility in determining the simulated food web. For example, you can specify the number of basal sources, baselines, and consumer species and families to simulate, as well as the sample size for each component and the trophic discrimination factors (TDFs) to use, among other things. The full list of arguments is:

- **n_sources**: Number of sources to simulate. Default to 3
- **source_C_range**: The range in source $\delta^{13}\text{C}$. A numeric vector of length 2 (minimum and maximum). Ignored if **source_C_mean** is provided
- **source_N_range**: The range in source $\delta^{15}\text{N}$. A numeric vector of length 2 (minimum and maximum). Ignored if **source_N_mean** is provided
- **source_C_mean**: Numeric vector of length **n_sources** with the mean $\delta^{13}\text{C}$ for each source. If NULL (default), mean $\delta^{13}\text{C}$ values are sampled from equal (**n_sources** * 2) intervals along the given **source_C_range**
- **source_N_mean**: Numeric vector of length **n_sources** with the mean $\delta^{15}\text{N}$ for each source. If NULL (default), mean $\delta^{15}\text{N}$ values are sampled from equal (**n_sources** * 2) intervals along the given **source_N_range**
- **source_C_sd**: Numeric vector of length **n_sources** with the $\delta^{13}\text{C}$ standard deviation for each source
- **source_N_sd**: Numeric vector of length **n_sources** with the $\delta^{15}\text{N}$ standard deviation for each source
- **n_obs_sources**: Numeric vector of length **n_sources** with the number of observations for each source. Can be a single value if all source have the same number of observations. Default to 10
- **n_baselines**: Number of baselines (i.e. primary consumers, TP = 2) to simulate. Must be 1 (default) or 2
- **n_obs_baselines**: Numeric vector of length **n_baselines** with number of observations for each base-

line. Can be a single value if all baselines have the same number of observations. Default to 10

- **n_families**: Number of consumer families to simulate. Default to 3
- **n_species**: Number of consumer species to simulate. Default to 10. These are randomly split among the families
- **n_obs_consumers**: Numeric vector of length **n_species** with number of observations for each consumer species. Can be a single value if all consumers have the same number of observations. Default to 10
- **sd_families**: Variation in diet across families. Default to 1.5
- **sd_species**: Variation in diet across species. Default to 0.5
- **TP_sd**: Standard deviation of trophic position for baselines and consumer species. Default to 0.05
- **n_TDF_C**: Number of observations for carbon's trophic discrimination factor
- **mean_TDF_C**: Mean for carbon's trophic discrimination factor
- **sd_TDF_C**: Standard deviation for carbon's trophic discrimination factor
- **n_TDF_N**: Number of observations for nitrogen's trophic discrimination factor
- **mean_TDF_N**: Mean for nitrogen's trophic discrimination factor
- **sd_TDF_N**: Standard deviation for nitrogen's trophic discrimination factor
- **C_baselines, C_consumers**: Total consumption of baselines and consumers (i.e. total number of source items eaten in the tissue turnover period). Default to 20
- **seed**: The seed passed to **base::set.seed()** to make simulations reproducible. If NA (the default), **set.seed** will not be called

We provide the function with a realistic range of $\delta^{13}\text{C}$ and $\delta^{15}\text{N}$ for basal sources based on those in the Mo'orea case study and randomly sample the source standard deviations from a uniform distribution, with a larger range for carbon than for nitrogen, as often observed. Additionally, we randomly sample the number of observations for each basal source, baseline and consumer species. We allow sample sizes of basal sources and baselines to range between 3 and 10, while for consumers we allow sample sizes as low as 1 to test if source contributions can be accurately recovered for sparsely sampled species.

We also need to set the total consumption (C) of baselines and consumers, which represents the number of times that they sample basal source distributions. Thus, C is inversely proportional to the resulting consumer isotopic variability. We set C to 10 for the baselines to reflect greater variability in lower trophic positions, and 40 for consumers, which results in reasonable standard deviations. To see how C influences consumer variability, rerun the following chunk of code by increasing or decreasing C (for instance set **C_consumers** = 5 or **C_consumers** = 100) and visualise the simulated food webs. Consumer's isotopic signatures simulated with $C = 100$ will have much smaller standard deviations than those simulated with $C = 5$.

Finally, we use default isotope-specific TDFs which are generalist, non-tissue-specific TDFs taken from Post (2002).

```
# Set seed
set.seed(my_seed)

# Sample source SDs
source_C_sd <- runif(3, 0.5, 2)
source_N_sd <- runif(3, 0.5, 1)

# Sample number of observations
n_obs_sources <- sample(3:10, 3, replace = TRUE)
n_obs_baselines <- sample(3:10, 2, replace = TRUE)
n_obs_consumers <- sample(1:10, 20, replace = TRUE)

# Simulate food web
sim_data <- simulate_food_web(n_sources = 3,
                             source_C_range = c(-20, -5),
                             source_N_range = c(0, 5),
                             source_C_sd = source_C_sd,
                             source_N_sd = source_N_sd,
```

```

n_baselines = 2,
n_families = 5,
n_species = 20,
n_obs_sources = n_obs_sources,
n_obs_baselines = n_obs_baselines,
n_obs_consumers = n_obs_consumers,
C_baselines = 10,
C_consumers = 40,
seed = my_seed)

```

The output of `simulate_food_web()` is a list with the following elements:

- `isotopes`: data frame with isotope data for all observations
- `sources_summary`: data frame with isotopic means and standard deviations for each source
- `TP_families`: data frame with number of species, mean and standard deviation of trophic position for each family
- `TP_species`: data frame with trophic position for each species
- `alpha_baselines`: data frame with source proportions for each baseline
- `alpha_global`: data frame with global source proportions
- `alpha_families`: data frame with source proportions for each family
- `alpha_species`: data frame with source proportions for each species
- `TDF`: list of length two including the simulated values of carbon and nitrogen trophic discrimination factors
- `TDF_summary`: data frame with mean and standard deviation of trophic discrimination factors for each source
- `isospace_plot`: ggplot object displaying the food web in isotopic space

```
str(sim_data, max.level = 1, give.attr = FALSE)
```

```

List of 11
 $ isotopes      : 'data.frame':   144 obs. of  6 variables:
 $ sources_summary: 'data.frame':    3 obs. of  5 variables:
 $ TP_families   : tibble [5 x 4] (S3: tbl_df/tbl/data.frame)
 $ TP_species    : tibble [20 x 3] (S3: tbl_df/tbl/data.frame)
 $ alpha_baselines: tibble [6 x 4] (S3: tbl_df/tbl/data.frame)
 $ alpha_global  : 'data.frame':    1 obs. of  3 variables:
 $ alpha_families: tibble [15 x 4] (S3: tbl_df/tbl/data.frame)
 $ alpha_species : tibble [60 x 5] (S3: tbl_df/tbl/data.frame)
 $ TDF           :List of 2
 $ TDF_summary   : 'data.frame':    3 obs. of  5 variables:
 $ isospace_plot :List of 11

```

We can then visualise the simulated food web in isotopic space by looking at the generated `isospace_plot`.

```

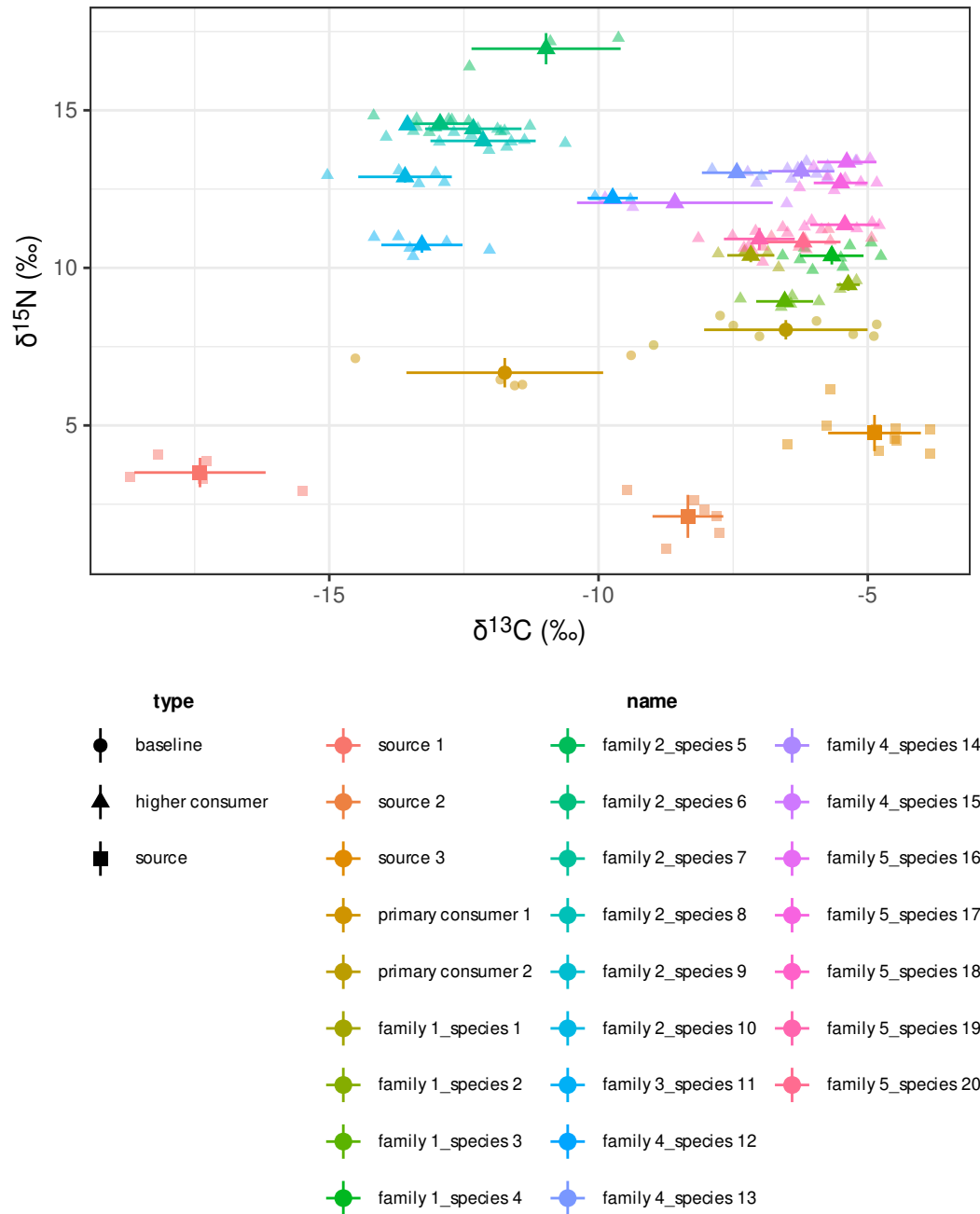
# We adjust the legend position and size to fit in the document
sim_data$isospace_plot +
  guides(shape = guide_legend(ncol = 1),
         color = guide_legend(ncol = 3)) +
  theme(legend.position = "bottom",
        legend.title.position = "top",
        legend.title = element_text(face = "bold", size = 8, hjust = 0.5),
        legend.text = element_text(size = 7))

```

```
Warning: Removed 1 row containing missing values or values outside the scale range
(`geom_segment()`).
```

```
Removed 1 row containing missing values or values outside the scale range
```

```
(`geom_segment()`).
```



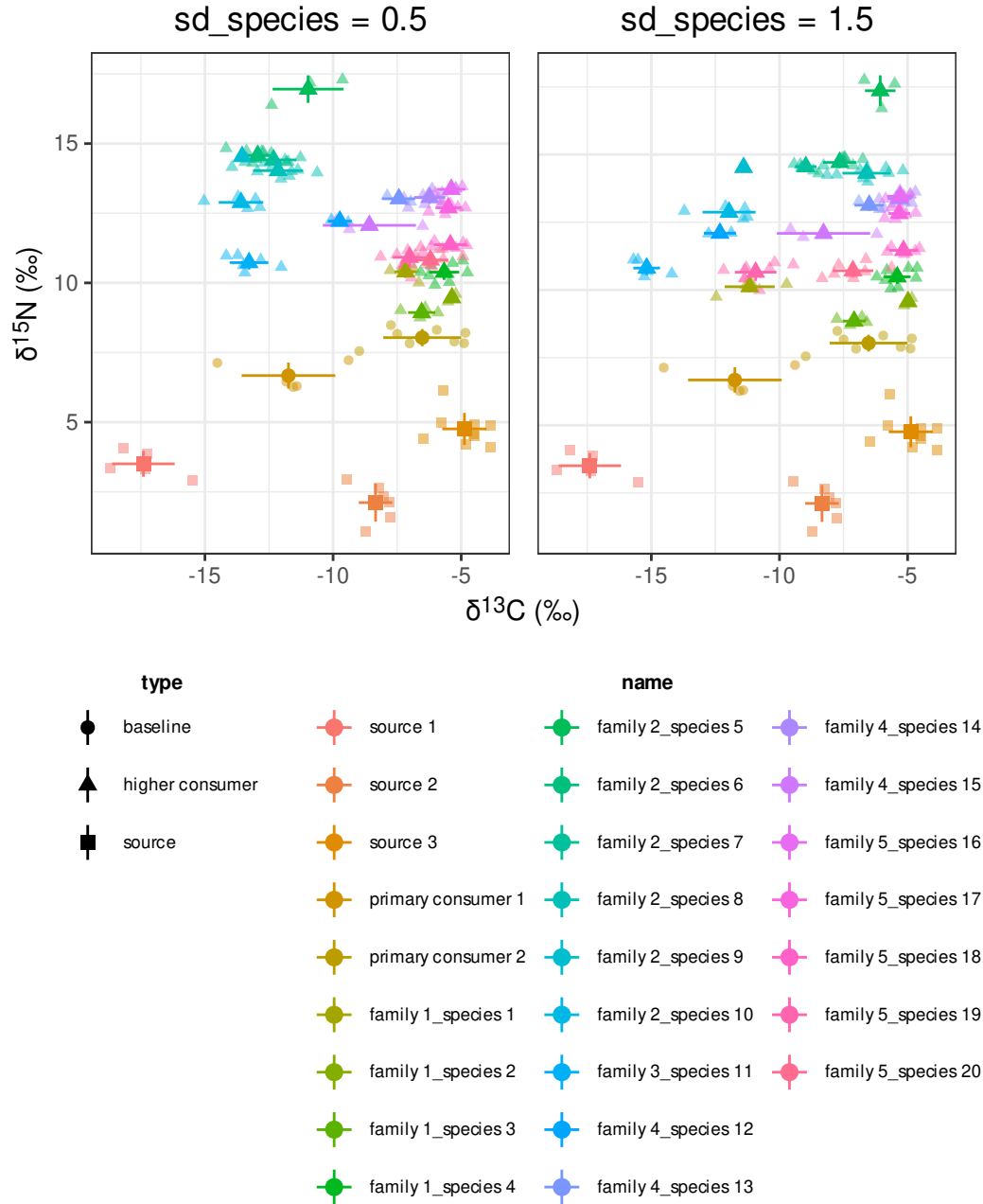
The warning indicates that one consumer species have only one observation and the error bars for this species (i.e. species 9) cannot be displayed.

Note that consumer families differ in $\delta^{15}\text{N}$ as they have trophic positions ranging between 2.64 and 4.21.

```
# Print family TP
sim_data$TP_families |>
  mutate(across(where(is.double), ~ round(.x, 3)))
```

family	n_species	TP_mean	TP_sd
family 1	4	2.636	0.203
family 2	6	4.206	0.259
family 3	1	3.013	0.131
family 4	4	3.634	0.138
family 5	5	3.157	0.230

Furthermore, related species (i.e. from the same family) are clustered together. This is because we used the default standard deviations for family and species, which are 1.5 and 0.5, respectively (see above). This simulates strong phylogenetic conservatism in consumers diet, where diet is more variable across than within families, which is a common feature in ecology. Increasing the species standard deviation, for example up to 1.5, results in a weaker phylogenetic conservatism and therefore greater variability in basal source use within each family, as can be observed by the greater dispersion of related species in the isotopic space below.



Since we simulated the data, there is no need to perform any preliminary data preparation steps and we can move directly to estimating the trophic position of consumers.

4 Estimating consumer trophic position

To estimate consumer trophic position we fit a two-baseline model to each species using the R package [tRophicPosition](#) (Quezada-Romegialli et al., 2018). To do this, we first need to extract the isotope data for each consumer species with the `extractIsotopeData()` function.

```
# Extract data for each consumer species
si_data_list <- extractIsotopeData(sim_data$isotopes |>
```

```
# remove sources
filter(type != "source"),
b1 = "primary consumer 1",
b2 = "primary consumer 2",
baselineColumn = "name",
consumersColumn = "name",
groupsColumn = NULL,
d13C = "d13C",
d15N = "d15N",
deltaC = sim_data$TDF$deltaC,
deltaN = sim_data$TDF$deltaN)
```

Then, we can estimate the trophic position using the `multiSpeciesTP()` function and the “twoBaselinesFull” model. We set `lambda` (i.e. the trophic position of baselines) to 2 because our isotopic baselines are primary consumers.

i Note

The results of this step, and consequently subsequent ones, will be slightly different each time the code is run. To obtain reproducible results, modifications to the `tRophicPosition::TPmodel()` source code are required.

```
# Fit multispecies twoBaselinesFull model with default arguments
estimated_TP <- multiSpeciesTP(siDataList = si_data_list,
                              lambda = 2,
                              model = "twoBaselinesFull")

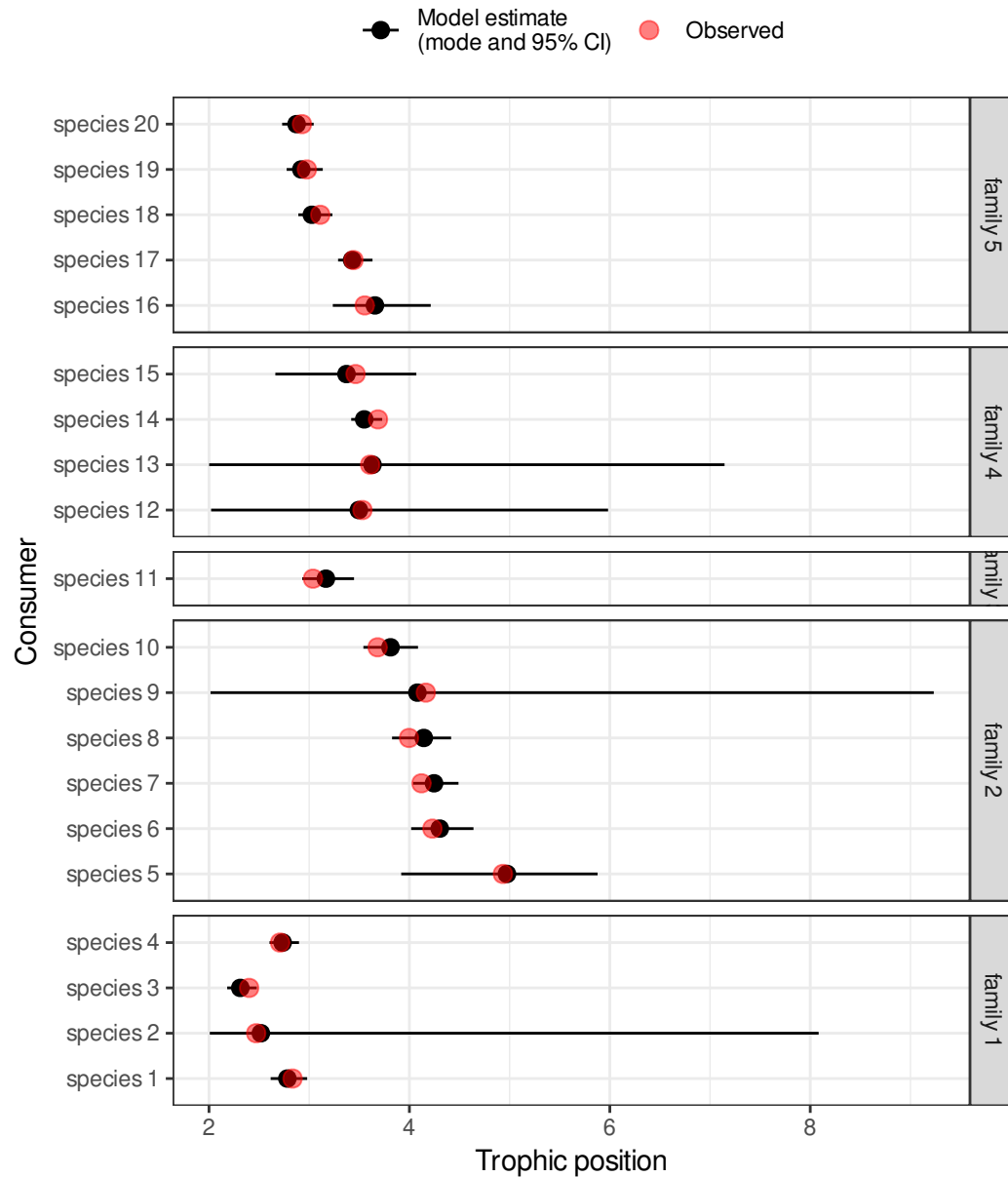
# Print estimated TP
estimated_TP$df |>
  select(-c(model, group, combined)) |>
  mutate(across(where(is.double), ~ round(.x, 3)))
```

consumer	lower	upper	median	mode	alpha.lower	alpha.upper	alpha.median	alpha.mode
family 1_species 1	2.616	2.980	2.789	2.782	0.001	0.475	0.192	0.172
family 1_species 2	2.008	8.084	2.739	2.518	0.001	0.887	0.237	0.038
family 1_species 3	2.182	2.477	2.320	2.313	0.000	0.372	0.103	0.019
family 1_species 4	2.604	2.898	2.739	2.735	0.000	0.349	0.060	0.011
family 2_species 5	3.918	5.878	4.950	4.973	0.221	1.000	0.759	0.914
family 2_species 6	4.018	4.639	4.310	4.303	0.640	1.000	0.913	0.987
family 2_species 7	4.038	4.490	4.260	4.246	0.612	1.000	0.892	0.981
family 2_species 8	3.827	4.416	4.140	4.144	0.687	1.000	0.908	0.977
family 2_species 9	2.015	9.233	4.886	4.080	0.063	0.998	0.562	0.883
family 2_species 10	3.541	4.086	3.812	3.812	0.618	1.000	0.908	0.991
family 3_species 11	2.930	3.447	3.163	3.167	0.665	1.000	0.914	0.987
family 4_species 12	2.021	5.983	3.517	3.496	0.147	0.998	0.635	0.677
family 4_species 13	2.002	7.145	3.690	3.632	0.001	0.871	0.346	0.278
family 4_species 14	3.419	3.728	3.556	3.549	0.000	0.327	0.103	0.042
family 4_species 15	2.662	4.069	3.398	3.372	0.047	0.941	0.475	0.406
family 5_species 16	3.236	4.213	3.674	3.656	0.000	0.744	0.131	0.021
family 5_species 17	3.288	3.632	3.437	3.428	0.000	0.393	0.071	0.016
family 5_species 18	2.891	3.232	3.039	3.027	0.000	0.433	0.073	0.011
family 5_species 19	2.775	3.135	2.935	2.921	0.000	0.477	0.172	0.143
family 5_species 20	2.730	3.045	2.878	2.874	0.000	0.349	0.087	0.012

The table shows summary statistics (median, mode, lower and upper bound of 95% CI) for the trophic position (first four columns) and the proportional contribution of baseline one (columns starting with “alpha”) estimated for each species.

We can compare the estimated and true (known) trophic position of each consumer species.

```
# Plot observed and estimated trophic positions
estimated_TP$df |>
  # split consumer name into family and species to plot a panel per family
  separate_wider_delim(consumer,
                        delim = "_",
                        names = c("family", "species")) |>
  # add name for legend
  mutate(TP_type = "Model estimate\n(mode and 95% CI)") |>
  ggplot(aes(x = mode, y = species, color = TP_type)) +
  geom_pointrange(aes(xmin = lower, xmax = upper)) +
  # add true TP
  geom_point(data = sim_data$TP_species |>
             # add name for legend
             mutate(TP_type = "Observed"),
             aes(x = TP),
             size = 3,
             alpha = 0.5) +
  scale_color_manual(values = c("Black", "Red")) +
  # split by family and reverse order
  facet_grid(rows = vars(fct_rev(family)),
             scales = "free",
             space = "free") +
  labs(x = "Trophic position",
       y = "Consumer") +
  theme_bw() +
  theme(legend.title = element_blank(),
        legend.position = "top")
```



The trophic position is relatively well estimated for all species. Species with low sample sizes have large uncertainties in estimated trophic position, although the point estimate (i.e. the mode) is close to the true value. The absolute error between the estimated mode and the true trophic position is:

```
estimated_TP$df |>
  separate_wider_delim(consumer,
                        delim = "_",
                        names = c("family", "species")) |>
  select(family, species, estimated_TP = mode) |>
  left_join(sim_data$TP_species |>
            rename(true_TP = TP)) |>
  mutate(absolute_error = abs(estimated_TP - true_TP),
         across(where(is.double), ~ round(.x, 3)))
```

family	species	estimated_TP	true_TP	absolute_error
family 1	species 1	2.782	2.834	0.052
family 1	species 2	2.518	2.471	0.047
family 1	species 3	2.313	2.401	0.088
family 1	species 4	2.735	2.710	0.025
family 2	species 5	4.973	4.933	0.040
family 2	species 6	4.303	4.230	0.073
family 2	species 7	4.246	4.121	0.125
family 2	species 8	4.144	3.996	0.147
family 2	species 9	4.080	4.164	0.084
family 2	species 10	3.812	3.683	0.129
family 3	species 11	3.167	3.038	0.129
family 4	species 12	3.496	3.531	0.036
family 4	species 13	3.632	3.608	0.023
family 4	species 14	3.549	3.686	0.137
family 4	species 15	3.372	3.463	0.091
family 5	species 16	3.656	3.554	0.102
family 5	species 17	3.428	3.442	0.014
family 5	species 18	3.027	3.110	0.083
family 5	species 19	2.921	2.977	0.055
family 5	species 20	2.874	2.928	0.054

Finally, we can compare the observed error with the expected error in trophic position for each species, obtained using the custom function `estimate_tp_error()`. This function has the following arguments:

- **sources:** A data frame with mean isotopic values of sources. Must have a column named “d15N_mean” and, if using two baselines, a column named “d13C_mean”.
- **baselines:** A data frame with mean isotopic values of one or two baselines. Must have a column named “d15N_mean” and, if using two baselines, a column named “d13C_mean”.
- **TP_b:** A numeric value. Trophic position of baseline(s). Default to 2 (i.e., primary consumers).
- **TDF_N:** A numeric value. The TDF for $\delta^{15}\text{N}$.
- **TDF_C:** A numeric value. The TDF for $\delta^{13}\text{C}$. Default to NULL. Only necessary for two baselines.
- **consumer_p:** A numeric vector summing to 1. The source relative contributions to the consumer.

```
# Prepare data for `estimate_tp_error()`
# Source means
source_means <- sim_data$sources_summary

# Baseline means
baseline_means <- sim_data$isotopes |>
  filter(type == "baseline") |>
  group_by(name) |>
  summarise(d13C_mean = mean(d13C),
            d15N_mean = mean(d15N))

# Source relative contributions to each species
consumers_p <- sim_data$alpha_species |>
  split(~species) |> # split by species
  purrr::map(~pull(.x, alpha)) # extract source contributions per species

# Compute expected error in TP for each species
expected_error <- purrr::map(
  consumers_p,
```

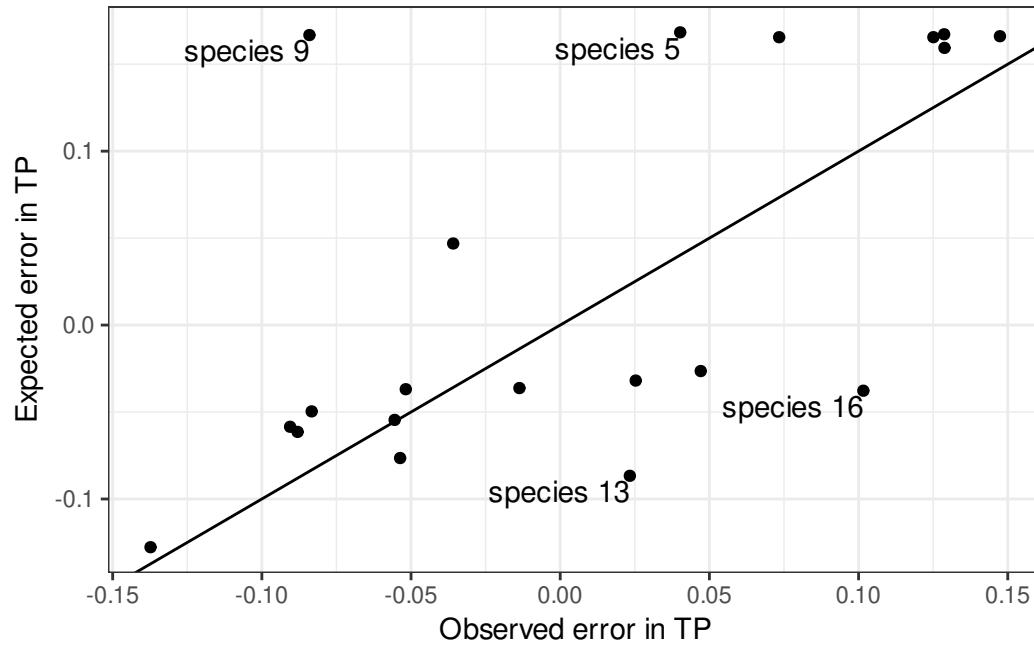
```

~ tibble(
  estimated_error = estimate_tp_error(sources = source_means,
                                     baselines = baseline_means,
                                     TP_b = 2,
                                     TDF_N = mean(sim_data$TDF$deltaN),
                                     TDF_C = mean(sim_data$TDF$deltaC),
                                     consumer_p = .x)
) |>
bind_rows(.id = "species")

# Compute observed error in TP for each species (as above but not absolute value)
observed_error <- estimated_TP$df |>
  separate_wider_delim(consumer,
                       delim = "_",
                       names = c("family", "species")) |>
  select(family, species, estimated_TP = mode) |>
  left_join(sim_data$TP_species |>
    rename(true_TP = TP),
    by = c("family", "species")) |>
  mutate(error = estimated_TP - true_TP)

# Plot relationship
observed_error |>
  left_join(expected_error, by = "species") |>
  mutate(diff = abs(error - estimated_error)) |>
  ggplot(aes(x = error, y = estimated_error)) +
  geom_point() +
  geom_abline() + # add 1:1 line
  geom_text(aes(label = ifelse(diff > 0.1, species, "")),
            hjust = 1, vjust = 1.2) + # add label for few points
  labs(x = "Observed error in TP",
       y = "Expected error in TP") +
  theme_bw()

```



We see that the error between the estimated and the true trophic position (observed error) is in line with the expected error in trophic position based on the source and baseline mean isotopic signatures, source relative contributions to consumers and the TDFs used ($r_{Pearson} = 0.65$). For a few species the error is further from the expected one (i.e. difference greater than 0.1) due to their very small sample size ($n = 3$ for species 5, $n = 1$ for species 9, $n = 2$ for species 13, $n = 3$ for species 16), which leads to a high uncertainty in the estimation of trophic position.

5 Isotopic rescaling of consumers

After estimating consumer trophic position, we can rescale consumer isotopic signatures to the food web base (i.e. $TP = 1$) by subtracting $TP - 1$ times the carbon and nitrogen TDFs. It would be preferable to propagate the uncertainty in TDFs and in the estimated consumer trophic positions into the rescaled isotopic signatures to obtain a more realistic uncertainty in basal source proportions. However, it is unfeasible because the inclusion of the measurement error in consumer isotopic signatures is not supported by existing Bayesian stable isotope mixing models. Therefore, we only use the mean TDFs and the estimated mode of trophic position.

```
# Get mean TDF values for C and N
mean_TDF_C <- mean(sim_data$TDF$deltaC)
mean_TDF_N <- mean(sim_data$TDF$deltaN)

# Rescale consumer's isotopic signatures
consumers_rescaled <- sim_data$isotopes |>
  filter(type == "higher consumer") |>
  left_join(estimated_TP$df |>
    select(consumer, TP = mode),
    by = c("name" = "consumer")) |>
  mutate(d13C = d13C - mean_TDF_C * (TP - 1),
    d15N = d15N - mean_TDF_N * (TP - 1)) |>
  select(family, species, d13C, d15N, TP)
```

6 Mixing model validation

Once consumer isotopic signatures are rescaled, we need to plot them in isotopic space to verify that they fall within the mixing polygon defined by the basal sources. We can do this using the function `plot_data()` from the R package `MixSIAR` (Stock et al., 2018). However, this function, and all other `MixSIAR` functions, requires data in a specific format. The package provides special functions for this purpose, which load source, TDF, and consumer data from CSV files. Therefore, we save the data in the format required by `MixSIAR` in temporary CSV files. Importantly, we set TDFs to zero because we have already accounted for the total trophic discrimination between the base of the food web and the consumers during the rescaling process.

```
# Sources (raw data): first column source names, then isotopes
sourcespath <- tempfile(pattern = "sim_sources", fileext = ".csv")
write_csv(sim_data$isotopes |>
  filter(type == "source") |>
  select(source = name, d13C, d15N),
  file = sourcespath)
# TDF: first column source names, then mean and sd of isotopes
# We set TDFs to 0 because we have rescaled consumers to the food web base
tdfpath <- tempfile(pattern = "sim_tdf", fileext = ".csv")
write_csv(sim_data$TDF_summary |>
  mutate(across(where(is.double), ~ 0)),
  file = tdfpath)
# Mixtures: data frame including isotopes and covariates
# We need family and species as covariates
consumerspath <- tempfile(pattern = "sim_consumers", fileext = ".csv")
write_csv(consumers_rescaled,
  file = consumerspath)
```

Then, we load the data with the special functions provided by `MixSIAR`. First, we load consumer data (i.e. mixtures) using the function `load_mix_data()`, because the resulting object is required to load source and TDF data. This function allows to define the covariates to be included in the model. `MixSIAR` is limited to one continuous covariate and two factors, that can be either fixed or random effects, and random effects can be nested. Since we have multiple observations per species, and species are grouped into families, we specify two random effects, family and species, with species nested within family.

```
# Mixtures: nested random effect of species within family
mix <- load_mix_data(consumerspath,
  iso_names = c("d13C", "d15N"),
  factors = c("family", "species"),
  fac_random = c(TRUE, TRUE),
  fac_nested = c(FALSE, TRUE),
  cont_effects = NULL)
```

By passing this object to the functions `load_source_data()` and `load_discr_data()`, we load the source and TDF data, respectively. Furthermore, we specify that source data are “raw” values.

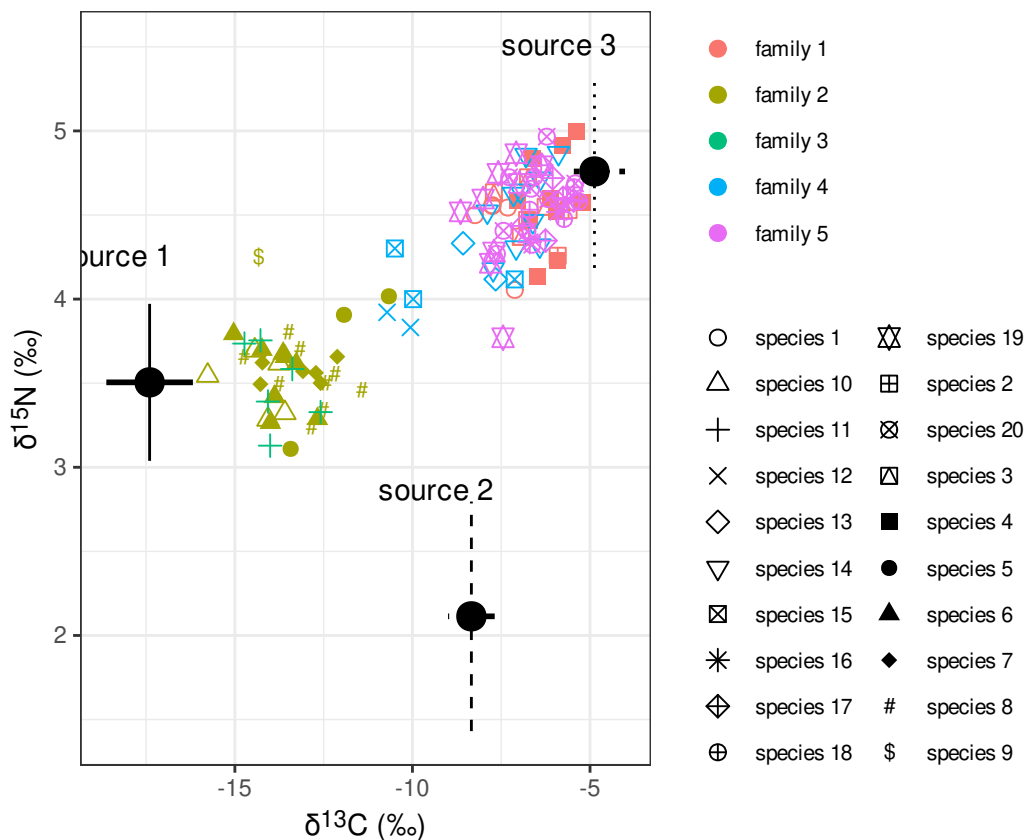
```
# Sources: raw data
source <- load_source_data(sourcespath,
  data_type = "raw",
  mix = mix,
  conc_dep = FALSE)
# TDF
tdf <- load_discr_data(tdfpath,
  mix = mix)
```

We can now plot the rescaled consumer data in isotopic space with the function `plot_data()`.

```
# Check isotopic space
p <- plot_data(mix = mix, source = source, discr = tdf,
  # avoid saving files
  plot_save_pdf = FALSE, plot_save_png = FALSE,
  return_obj = TRUE)

# Modify point shapes and legend position
p <- p +
  # Replace shape scale because MixSIAR allows a maximum of 15
  scale_shape_manual(values = c(1:18, 35, 36),
    labels = mix$FAC[[2]]$labels) +
  # Change legend position for clarity (default inside the plot)
  theme(legend.position = "right",
    legend.text = element_text(size = 8)) +
  # Make two columns
  guides(color = guide_legend(ncol = 1),
    shape = guide_legend(ncol = 2))

p
```



After rescaling the isotopic data, all consumers fall within the mixing polygon of the sources. Furthermore, we can see that families 1, 4 and 5 are close to source 3, while families 2 and 3 are closer to source 1. No much distinction can be noted among related species as we used a small standard deviation of 0.5.

Additionally, we can calculate the normalised surface area of the mixing polygon using the function `calc_area()` from MixSIAR.

```

polygon_area <- calc_area(source = source, mix = mix, discr = tdf)
polygon_area

```

```
[1] 27.47655
```

This value indicates that we have a well-resolved polygon with less than 20% bias according to Brett (2014). We can also use equations 3 and 4 from Brett (2014) and values from Table 1 of the same paper to obtain the actual bias for consumers dominated by a single source.

```

# Function to compute bias using eq 3 and 4 from Brett 2014
# Values are from table 1 in Brett 2014 for 3 resources
get_bias <- function(area) {
  y <- 0.803 + ((0.334 - 0.803) / (1 + exp((log(area) - 1.247) / 0.832)))

  bias <- (1 - ((y - 1/3) / (0.8 - 1/3))) * 100

  return(bias)
}

get_bias(area = polygon_area)

```

```
[1] 7.097433
```

Given our well-resolved polygon, we should obtain relatively accurate results from the stable isotope mixing model.

7 Estimating basal source proportions

To estimate basal source proportions for each consumer species, we fit a Bayesian hierarchical stable isotope mixing model using MixSIAR. First, we use the function `write_JAGS_model()` to write a JAGS model to a “txt” file as required by MixSIAR, specifying a process only error structure because we have a hierarchical structure and at least one consumer species with a single observation.

```

# Write model with process only error
modelpath <- tempfile(pattern = "test_model", fileext = ".txt")
write_JAGS_model(modelpath,
  resid_err = FALSE,
  process_err = TRUE,
  mix = mix,
  source = source)

```

Then, we fit the model using the function `run_model()`, with the default “uninformative”/generalist prior on global proportions (*Dirichlet*(1, 1, 1)), which can be shortened as `alpha.prior = 1`. Given the hierarchical structure, and thus many parameters to estimate, the model requires a large number of iterations to converge. We set `run = "long"`, which fits 3 chains, each with 300,000 iterations, a of 200,000-iteration burn-in and a thinning of 100, thus retaining 3,000 posterior draws (i.e., 1,000 x 3 chains).

```

# Fit model with "uninformative" alpha prior
mixsiar_fit <- run_model(run = "long",
  mix = mix,
  source = source,
  discr = tdf,
  model_filename = modelpath,
  alpha.prior = 1)

```

With this large number of iterations the model takes several minutes to run. Once finished, we can check if it has converged using the function `output_diagnostics()`. But first we set the options to save diagnostics

and summary statistics to files and avoid saving plots.

```
# Set output options
# Save output files in separate folder
output_dir <- "MixSIAR_output"
if (!dir.exists(output_dir)) dir.create(output_dir)
mixsiar_options <- list(
  summary_save = TRUE,
  summary_name = paste(output_dir, "summary_statistics", sep = "/"),
  sup_post = TRUE,
  plot_post_save_pdf = FALSE,
  plot_post_name = paste(output_dir, "posterior_density", sep = "/"),
  sup_pairs = TRUE,
  plot_pairs_save_pdf = FALSE,
  plot_pairs_name = paste(output_dir, "pairs_plot", sep = "/"),
  sup_xy = TRUE,
  plot_xy_save_pdf = FALSE,
  plot_xy_name = paste(output_dir, "xy_plot", sep = "/"),
  gelman = TRUE,
  heidel = FALSE,
  geweke = TRUE,
  diag_save = TRUE,
  diag_name = paste(output_dir, "diagnostics", sep = "/"),
  indiv_effect = FALSE,
  plot_post_save_png = FALSE,
  plot_pairs_save_png = FALSE,
  plot_xy_save_png = FALSE,
  diag_save_ggmcmc = FALSE,
  return_obj = TRUE
)

# Check diagnostics
model_diagnostics <- output_diagnostics(mixsiar_fit,
                                         mix = mix,
                                         source = source,
                                         output_options = mixsiar_options)
```

```
#####
# Gelman-Rubin Diagnostic
#####
```

Generally the Gelman diagnostic should be < 1.05

```
Out of 241 variables: 8 > 1.01
                    0 > 1.05
                    0 > 1.1
```

The worst variables are:

	Point est.	Upper C.I.
ilr.fac1[4,1]	1.027255	1.057910
p.global[2]	1.026542	1.060438
p.global[1]	1.023141	1.082546
ilr.fac1[1,1]	1.019040	1.055797
ilr.fac1[2,1]	1.015015	1.046581
loglik[90]	1.011011	1.018823

```
loglik[63]      1.010248  1.015013
ilr.fac1[5,1]  1.010222  1.023835
p.fac2[17,1]   1.009613  1.027314
loglik[91]     1.009392  1.017077
```

```
#####
# Geweke Diagnostic
#####
```

The Geweke diagnostic is a standard z-score, so we'd expect 5% to be outside ± 1.96
Number of variables outside ± 1.96 in each chain (out of 241):

```
Chain 1 Chain 2 Chain 3
Geweke      25      15      15
```

Diagnostics look good, indicating model convergence. Thus, we use the function `output_statistics()` to get a summary table with point estimates and credible intervals of each parameter.

```
# Get summary statistics
model_stats <- output_stats(mixsiar_fit,
                             mix = mix,
                             source = source,
                             output_options = mixsiar_options) |>
  as.data.frame() |>
  rownames_to_column("parameter")

model_stats
```

parameter	Mean	SD	2.5%	5%	25%	50%	75%	95%	97.5%
family.SD	1.062	0.402	0.560	0.611	0.803	0.970	1.229	1.787	2.048
species.SD	0.293	0.080	0.164	0.180	0.237	0.283	0.339	0.437	0.474
p.global.source 1	0.356	0.115	0.154	0.179	0.277	0.348	0.432	0.557	0.599
p.global.source 2	0.140	0.082	0.025	0.035	0.081	0.124	0.181	0.300	0.338
p.global.source 3	0.504	0.124	0.270	0.302	0.416	0.503	0.592	0.706	0.746
p.family 1.source 1	0.134	0.036	0.071	0.080	0.108	0.132	0.156	0.196	0.211
p.family 2.source 1	0.636	0.050	0.541	0.555	0.603	0.636	0.668	0.716	0.735
p.family 3.source 1	0.640	0.097	0.434	0.473	0.578	0.646	0.708	0.790	0.811
p.family 4.source 1	0.273	0.051	0.178	0.194	0.239	0.271	0.304	0.357	0.379
p.family 5.source 1	0.126	0.031	0.068	0.078	0.104	0.124	0.145	0.180	0.191
p.family 1.source 2	0.075	0.048	0.005	0.010	0.038	0.066	0.102	0.164	0.186
p.family 2.source 2	0.148	0.056	0.047	0.061	0.110	0.145	0.184	0.243	0.265
p.family 3.source 2	0.152	0.093	0.013	0.026	0.084	0.141	0.206	0.322	0.369
p.family 4.source 2	0.088	0.054	0.008	0.013	0.048	0.080	0.119	0.187	0.211
p.family 5.source 2	0.065	0.042	0.005	0.009	0.035	0.059	0.090	0.143	0.167
p.family 1.source 3	0.791	0.048	0.687	0.709	0.762	0.793	0.824	0.866	0.878
p.family 2.source 3	0.216	0.047	0.131	0.143	0.185	0.214	0.245	0.295	0.315
p.family 3.source 3	0.208	0.085	0.069	0.087	0.148	0.198	0.259	0.364	0.404
p.family 4.source 3	0.639	0.060	0.515	0.538	0.602	0.643	0.681	0.729	0.744
p.family 5.source 3	0.809	0.042	0.715	0.734	0.783	0.812	0.837	0.874	0.884
p.species 1.source 1	0.175	0.042	0.092	0.105	0.146	0.176	0.203	0.243	0.257
p.species 10.source 1	0.695	0.049	0.599	0.614	0.662	0.695	0.728	0.775	0.791
p.species 11.source 1	0.675	0.056	0.569	0.588	0.637	0.673	0.713	0.769	0.789
p.species 12.source 1	0.379	0.060	0.259	0.280	0.339	0.379	0.419	0.479	0.499

(continued ...)

(continued)

parameter	Mean	SD	2.5%	5%	25%	50%	75%	95%	97.5%
p.species 13.source 1	0.252	0.053	0.147	0.167	0.218	0.251	0.287	0.338	0.359
p.species 14.source 1	0.167	0.030	0.109	0.118	0.146	0.167	0.186	0.216	0.224
p.species 15.source 1	0.310	0.049	0.210	0.226	0.279	0.310	0.343	0.392	0.407
p.species 16.source 1	0.113	0.035	0.049	0.059	0.089	0.111	0.136	0.173	0.189
p.species 17.source 1	0.107	0.031	0.050	0.059	0.085	0.107	0.128	0.159	0.173
p.species 18.source 1	0.103	0.030	0.047	0.055	0.082	0.101	0.122	0.154	0.165
p.species 19.source 1	0.173	0.034	0.104	0.114	0.150	0.173	0.196	0.227	0.237
p.species 2.source 1	0.113	0.038	0.048	0.056	0.085	0.110	0.137	0.179	0.194
p.species 20.source 1	0.129	0.031	0.069	0.077	0.107	0.129	0.150	0.182	0.192
p.species 3.source 1	0.137	0.038	0.066	0.076	0.111	0.137	0.163	0.200	0.210
p.species 4.source 1	0.103	0.027	0.052	0.058	0.085	0.103	0.122	0.149	0.160
p.species 5.source 1	0.567	0.062	0.448	0.469	0.525	0.566	0.608	0.671	0.690
p.species 6.source 1	0.663	0.043	0.581	0.595	0.635	0.662	0.691	0.735	0.750
p.species 7.source 1	0.622	0.047	0.527	0.545	0.589	0.624	0.654	0.698	0.713
p.species 8.source 1	0.606	0.044	0.522	0.534	0.575	0.606	0.636	0.680	0.693
p.species 9.source 1	0.668	0.067	0.534	0.556	0.623	0.669	0.712	0.781	0.804
p.species 1.source 2	0.084	0.056	0.006	0.011	0.041	0.074	0.115	0.192	0.216
p.species 10.source 2	0.134	0.055	0.036	0.048	0.094	0.130	0.170	0.232	0.251
p.species 11.source 2	0.146	0.078	0.013	0.023	0.087	0.143	0.199	0.281	0.309
p.species 12.source 2	0.102	0.067	0.008	0.013	0.051	0.091	0.142	0.225	0.258
p.species 13.source 2	0.091	0.061	0.007	0.012	0.045	0.080	0.126	0.204	0.233
p.species 14.source 2	0.068	0.041	0.006	0.011	0.037	0.063	0.092	0.143	0.161
p.species 15.source 2	0.091	0.059	0.008	0.013	0.047	0.083	0.125	0.204	0.230
p.species 16.source 2	0.065	0.045	0.004	0.008	0.033	0.056	0.089	0.154	0.179
p.species 17.source 2	0.063	0.042	0.004	0.008	0.031	0.055	0.088	0.144	0.164
p.species 18.source 2	0.061	0.041	0.004	0.008	0.030	0.054	0.082	0.140	0.165
p.species 19.source 2	0.072	0.046	0.005	0.010	0.037	0.064	0.099	0.157	0.180
p.species 2.source 2	0.073	0.052	0.005	0.009	0.034	0.062	0.101	0.169	0.198
p.species 20.source 2	0.066	0.043	0.004	0.009	0.034	0.059	0.090	0.148	0.169
p.species 3.source 2	0.074	0.050	0.005	0.010	0.037	0.066	0.103	0.164	0.190
p.species 4.source 2	0.066	0.043	0.005	0.009	0.033	0.060	0.091	0.145	0.165
p.species 5.source 2	0.167	0.075	0.042	0.058	0.114	0.161	0.212	0.298	0.332
p.species 6.source 2	0.145	0.056	0.043	0.056	0.106	0.144	0.183	0.239	0.258
p.species 7.source 2	0.158	0.063	0.047	0.061	0.114	0.154	0.196	0.269	0.295
p.species 8.source 2	0.172	0.064	0.049	0.069	0.128	0.171	0.213	0.278	0.299
p.species 9.source 2	0.125	0.057	0.034	0.043	0.084	0.119	0.160	0.228	0.252
p.species 1.source 3	0.741	0.051	0.636	0.654	0.711	0.743	0.775	0.822	0.837
p.species 10.source 3	0.171	0.041	0.090	0.105	0.143	0.170	0.198	0.238	0.251
p.species 11.source 3	0.179	0.054	0.068	0.086	0.143	0.181	0.217	0.264	0.278
p.species 12.source 3	0.519	0.066	0.388	0.410	0.475	0.520	0.562	0.625	0.644
p.species 13.source 3	0.657	0.062	0.537	0.557	0.615	0.658	0.700	0.758	0.777
p.species 14.source 3	0.766	0.037	0.691	0.704	0.742	0.767	0.791	0.825	0.834
p.species 15.source 3	0.598	0.055	0.483	0.504	0.563	0.600	0.636	0.685	0.702
p.species 16.source 3	0.822	0.048	0.718	0.735	0.792	0.824	0.855	0.895	0.908
p.species 17.source 3	0.830	0.043	0.738	0.755	0.801	0.832	0.859	0.898	0.908
p.species 18.source 3	0.836	0.043	0.746	0.761	0.810	0.839	0.866	0.902	0.914
p.species 19.source 3	0.756	0.040	0.672	0.689	0.729	0.757	0.783	0.819	0.829
p.species 2.source 3	0.814	0.055	0.698	0.719	0.778	0.817	0.853	0.900	0.914
p.species 20.source 3	0.805	0.040	0.723	0.738	0.779	0.806	0.833	0.867	0.878
p.species 3.source 3	0.788	0.048	0.692	0.706	0.757	0.790	0.821	0.864	0.875

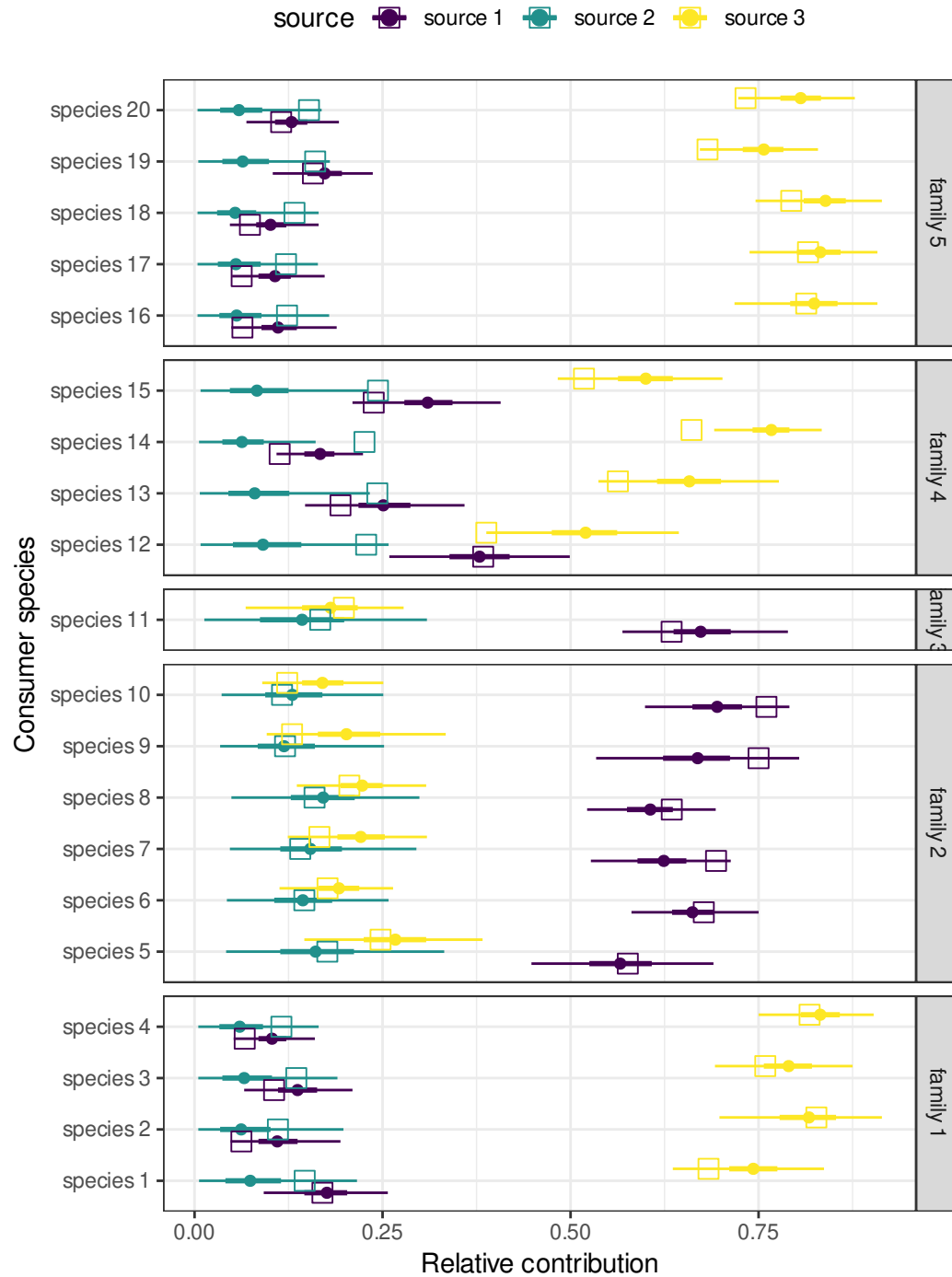
(continued ...)

(continued)

parameter	Mean	SD	2.5%	5%	25%	50%	75%	95%	97.5%
p.species 4.source 3	0.831	0.039	0.750	0.763	0.806	0.832	0.858	0.892	0.903
p.species 5.source 3	0.266	0.062	0.146	0.163	0.225	0.267	0.308	0.366	0.383
p.species 6.source 3	0.191	0.040	0.113	0.127	0.165	0.192	0.219	0.255	0.264
p.species 7.source 3	0.220	0.047	0.124	0.142	0.190	0.221	0.253	0.298	0.309
p.species 8.source 3	0.222	0.043	0.136	0.150	0.194	0.223	0.250	0.293	0.308
p.species 9.source 3	0.206	0.061	0.096	0.112	0.164	0.202	0.247	0.310	0.334

Now we can compare model estimates with true values of source proportions. For each consumer species and source we plot the posterior median, 50% and 95% CIs (points and intervals) along with the true values (empty squares).

```
# Plot species-level estimates
model_stats |>
  filter(startsWith(parameter, "p.species")) |>
  separate_wider_delim(parameter,
    delim = ".",
    names = c("p", "species", "source")) |>
  left_join(sim_data$alpha_species,
    by = c("species", "source")) |>
  mutate(species = factor(species, levels = paste("species", 1:20))) |>
  ggplot(aes(y = species, color = source)) +
  geom_linerange(aes(xmin = `2.5%`, xmax = `97.5%`),
    linewidth = 0.5,
    position = position_dodge(width = 0.7)) +
  geom_pointrange(aes(x = `50%`, xmin = `25%`, xmax = `75%`),
    linewidth = 1,
    fatten = 2,
    position = position_dodge(width = 0.7)) +
  geom_point(aes(x = alpha),
    position = position_dodge(width = 0.7),
    shape = 22,
    size = 4) +
  scale_color_viridis_d() +
  facet_grid(rows = vars(fct_rev(family)),
    scales = "free",
    space = "free") +
  labs(x = "Relative contribution",
    y = "Consumer species") +
  theme_bw() +
  theme(legend.position = "top")
```



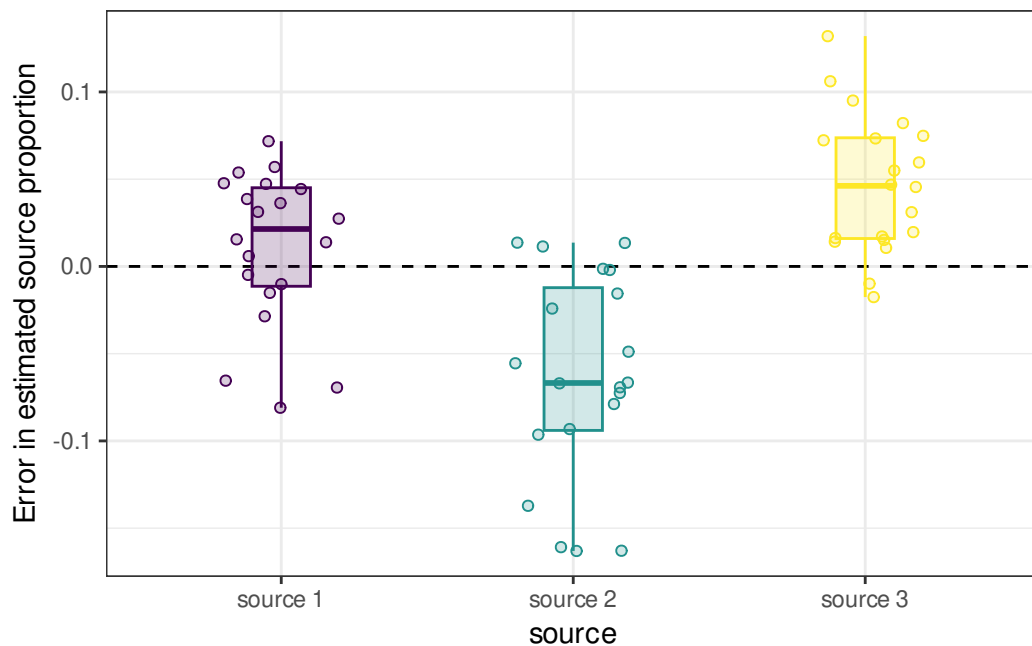
Overall, true source proportions were recovered relatively well, with nearly all values within the estimated 95% credible intervals, regardless of the number of observations. Furthermore, point estimates deviate less than 16% from the true proportions (mean = 5.19%) as shown in the figure below.

```
# Plot error in estimated source proportions
model_stats |>
  filter(startsWith(parameter, "p.species")) |>
  separate_wider_delim(parameter,
    delim = ".",
```

```

      names = c("p", "species", "source")) |>
left_join(sim_data$alpha_species,
          by = c("species", "source")) |>
mutate(error = `50%` - alpha) |>
ggplot(aes(x = source,
           y = error,
           color = source,
           fill = stage(source,
                        after_scale = alpha(fill, 0.2)))) +
  geom_hline(yintercept = 0, linetype = "dashed") +
  geom_boxplot(outliers = FALSE, width = 0.2) +
  geom_point(position = position_jitter(width = 0.2, seed = my_seed),
            shape = 21) +
  ylab("Error in estimated source proportion") +
  scale_color_viridis_d(aesthetics = c("color", "fill")) +
  theme_bw() +
  theme(legend.position = "none")

```



Finally, we can compare model estimates with true values of source proportions at the family and global levels and we can check whether the model was able to recover the simulated species and family standard deviations.

```

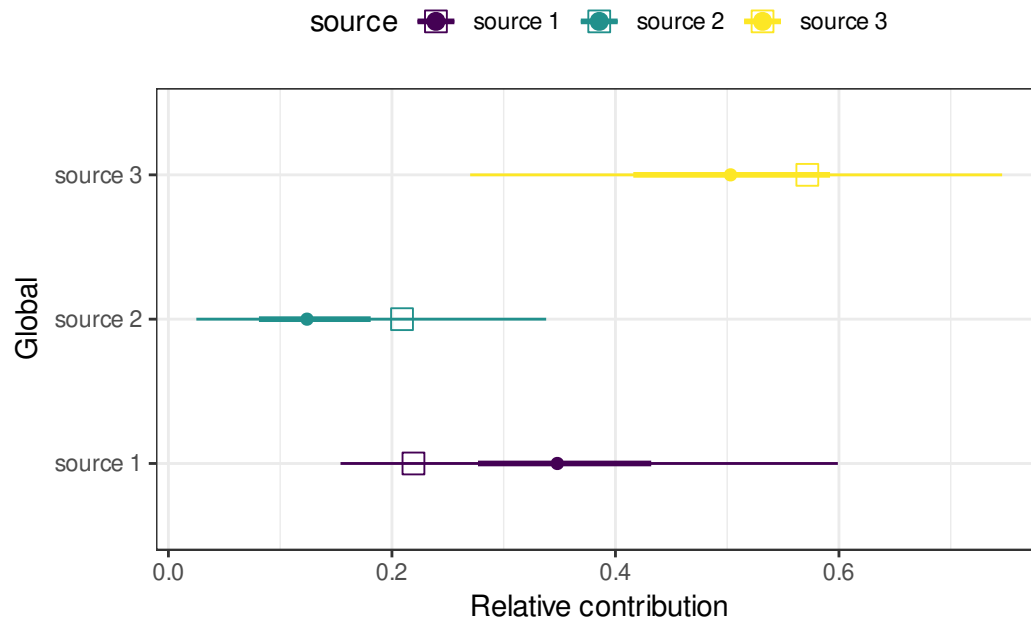
# Plot global estimates
model_stats |>
  filter(startsWith(parameter, "p.global")) |>
  separate_wider_delim(parameter,
                        delim = ".",
                        names = c("p", "global", "source")) |>
left_join(sim_data$alpha_global |>
  pivot_longer(cols = everything(),
               names_to = "source",
               values_to = "alpha")) |>

```

```

ggplot(aes(y = source, color = source)) +
  geom_linerange(aes(xmin = `2.5%`, xmax = `97.5%`),
    linewidth = 0.5,
    position = position_dodge(width = 0.7)) +
  geom_pointrange(aes(x = `50%`, xmin = `25%`, xmax = `75%`),
    linewidth = 1,
    fatten = 2,
    position = position_dodge(width = 0.7)) +
  geom_point(aes(x = alpha), shape = 22, size = 4) +
  scale_color_viridis_d() +
  labs(x = "Relative contribution",
    y = "Global") +
  theme_bw() +
  theme(legend.position = "top")

```



```

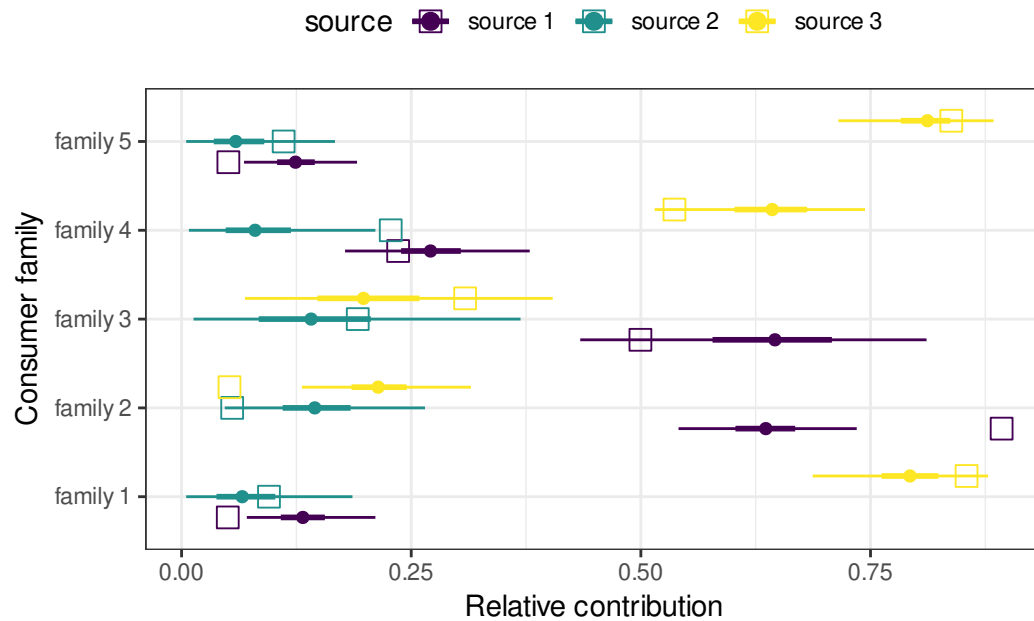
# Plot family-level estimates
model_stats |>
  filter(startsWith(parameter, "p.family")) |>
  separate_wider_delim(parameter,
    delim = ".",
    names = c("p", "family", "source")) |>
  left_join(sim_data$alpha_families) |>
  ggplot(aes(y = family, color = source)) +
  geom_linerange(aes(xmin = `2.5%`, xmax = `97.5%`),
    linewidth = 0.5,
    position = position_dodge(width = 0.7)) +
  geom_pointrange(aes(x = `50%`, xmin = `25%`, xmax = `75%`),
    linewidth = 1,
    fatten = 2,
    position = position_dodge(width = 0.7)) +
  geom_point(aes(x = alpha),
    position = position_dodge(width = 0.7),

```

```

    shape = 22,
    size = 4) +
  scale_color_viridis_d() +
  labs(x = "Relative contribution",
       y = "Consumer family") +
  theme_bw() +
  theme(legend.position = "top")

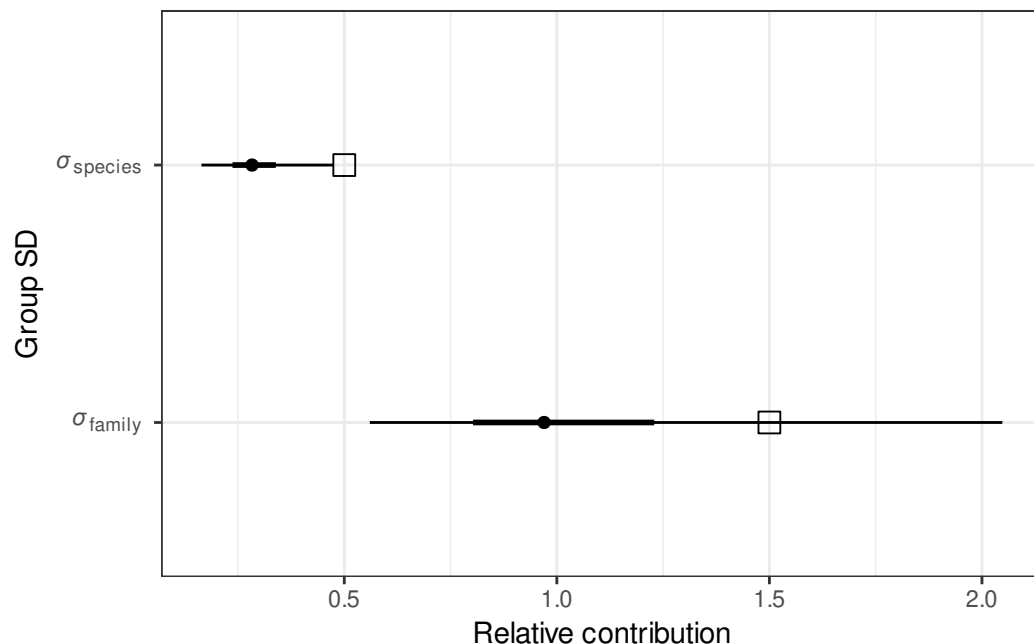
```



```

# Plot group SDs
model_stats |>
  filter(endsWith(parameter, "SD")) |>
  left_join(data.frame(parameter = c("species.SD", "family.SD"),
                          true_SD = c(0.5, 1.5)),
            by = "parameter") |>
  mutate(parameter = paste("<i>&sigma;</i><sub>",
                          gsub(".SD", "", parameter),
                          "</sub>")) |>
  ggplot(aes(y = parameter)) +
  geom_linerange(aes(xmin = `2.5%`, xmax = `97.5%`),
                linewidth = 0.5) +
  geom_pointrange(aes(x = `50%`, xmin = `25%`, xmax = `75%`),
                 linewidth = 1,
                 fatten = 2) +
  geom_point(aes(x = true_SD), shape = 22, size = 4) +
  labs(x = "Relative contribution",
       y = "Group SD") +
  theme_bw() +
  theme(axis.text.y = element_markdown())

```

We can see that the model did a relatively good job of recovering source proportions at the family and global levels, as well as standard deviations.

8 References

- Brett, M. T. (2014). Resource polygon geometry predicts bayesian stable isotope mixing model bias. *Marine Ecology Progress Series*, 514, 1–12. <https://doi.org/10.3354/meps11017>
- Post, D. M. (2002). Using stable isotopes to estimate trophic position: Models, methods, and assumptions. *Ecology*, 83(3), 703–718. [https://doi.org/10.1890/0012-9658\(2002\)083%5B0703:USITET%5D2.0.CO;2](https://doi.org/10.1890/0012-9658(2002)083%5B0703:USITET%5D2.0.CO;2)
- Quezada-Romegialli, C., Jackson, A. L., Hayden, B., Kahilainen, K. K., Lopes, C., & Harrod, C. (2018). tRophicPosition, an r package for the bayesian estimation of trophic position from consumer stable isotope ratios. *Methods in Ecology and Evolution*, 9(6), 1592–1599. <https://doi.org/10.1111/2041-210X.13009>
- Stock, B. C., Jackson, A. L., Ward, E. J., Parnell, A. C., Phillips, D. L., & Semmens, B. X. (2018). Analyzing mixing systems using a new generation of bayesian tracer mixing models. *PeerJ*, 6, e5096. <https://doi.org/10.7717/peerj.5096>

9 Computational environment

R version 4.4.2 (2024-10-31)

Platform: x86_64-pc-linux-gnu

Running under: Ubuntu 22.04.4 LTS

Matrix products: default

BLAS: /usr/lib/x86_64-linux-gnu/openblas-pthread/libblas.so.3

LAPACK: /usr/lib/x86_64-linux-gnu/openblas-pthread/libopenblas-p-r0.3.20.so; LAPACK version 3.10.0

locale:

[1] LC_CTYPE=en_GB.UTF-8	LC_NUMERIC=C
[3] LC_TIME=en_GB.UTF-8	LC_COLLATE=en_GB.UTF-8
[5] LC_MONETARY=en_GB.UTF-8	LC_MESSAGES=en_GB.UTF-8
[7] LC_PAPER=en_GB.UTF-8	LC_NAME=C

```
[9] LC_ADDRESS=C                LC_TELEPHONE=C
[11] LC_MEASUREMENT=en_GB.UTF-8 LC_IDENTIFICATION=C
```

```
time zone: Europe/Paris
tzcode source: system (glibc)
```

```
attached base packages:
```

```
[1] stats      graphics  grDevices  utils      datasets  methods    base
```

```
other attached packages:
```

```
[1] ggtext_0.1.2      patchwork_1.2.0      MixSIAR_3.1.12
[4] tRophicPosition_0.8.0 ggplot2_3.5.1        forcats_1.0.0
[7] tidyr_1.3.1       dplyr_1.1.4          tibble_3.2.1
[10] readr_2.1.5
```

```
loaded via a namespace (and not attached):
```

```
[1] gtable_0.3.5      tensorA_0.36.2.1     xfun_0.52
[4] lattice_0.22-6    tzdb_0.4.0           vctrs_0.6.5
[7] tools_4.4.2       generics_0.1.3       parallel_4.4.2
[10] DEoptimR_1.1-3-1  pkgconfig_2.0.3      Matrix_1.7-1
[13] lifecycle_1.0.4   compiler_4.4.2       farver_2.1.2
[16] stringr_1.5.1     MatrixModels_0.5-3   mcmc_0.9-8
[19] munsell_0.5.1     SparseM_1.84-2       quantreg_5.98
[22] htmltools_0.5.8.1 yaml_2.3.10          pillar_1.10.2
[25] crayon_1.5.2      MASS_7.3-61          boot_1.3-31
[28] abind_1.4-5       robustbase_0.99-4-1  commonmark_1.9.1
[31] tidyselect_1.2.1  digest_0.6.37        stringi_1.8.4
[34] reshape2_1.4.4    purrr_1.0.4          splines_4.4.2
[37] labeling_0.4.3    fastmap_1.2.0        grid_4.4.2
[40] colorspace_2.1-0  cli_3.6.5            magrittr_2.0.3
[43] compositions_2.0-8 survival_3.7-0        withr_3.0.2
[46] scales_1.3.0      sp_2.1-4             bit64_4.0.5
[49] rmarkdown_2.29    bit_4.0.5            rjags_4-16
[52] hms_1.1.3         kableExtra_1.4.0     coda_0.19-4.1
[55] evaluate_1.0.3    knitr_1.50           splancs_2.01-45
[58] EnvStats_3.0.0    viridisLite_0.4.2    markdown_1.12
[61] MCMCpack_1.7-1    R2jags_0.8-9         rlang_1.1.6
[64] gridtext_0.1.5    Rcpp_1.0.12          glue_1.8.0
[67] bayesm_3.1-6      xml2_1.3.6           reshape_0.8.9
[70] svglite_2.1.3     rstudioapi_0.16.0    vroom_1.6.5
[73] jsonlite_2.0.0    plyr_1.8.9           R6_2.6.1
[76] R2WinBUGS_2.1-22.1 systemfonts_1.1.0
```